ENGENHARIA DE SOFTWARE

GUILHERME FEITOSA ALVES - 50018205

PORTFÓLIO – RELATÓRIO DE AULA PRÁTICA:

TESTES E MANUTENÇÃO DE SOFTWARE

Milagres/CE
2026

GUILHERME FEITOSA ALVES - 50018205

PORTFÓLIO – RELATÓRIO DE AULA PRÁTICA

TESTES E MANUTENÇÃO DE SOFTWARE

Trabalho de portfólio apresentado como requisito parcial para a obtenção de pontos para a média semestral.

Orientador: Frederico Aparecido Faedo Pinto

Milagres/CE
2026

SUMÁRIO

1 INTRODUÇÃO.....3
2 DESENVOLVIMENTO.....4
2.1 MÉTODO.....4
2.2 RESULTADOS.....5
FIGURA 1 – 24/02/26 - Guilherme Feitosa Alves.....5
FIGURA 2 – 24/02/26 - Guilherme Feitosa Alves.....6
FIGURA 3 – 24/02/26 - Guilherme Feitosa Alves.....7
FIGURA 4 – 24/02/26 - Guilherme Feitosa Alves.....8
FIGURA 5 – 24/02/26 - Guilherme Feitosa Alves.....9
3 CONCLUSÃO.....10
REFERÊNCIAS.....11

1 INTRODUÇÃO

Este relatório apresenta as etapas de realização do portfólio da disciplina Testes e Manutenção de Software, cujo objetivo principal é realizar teste de caixa branca para comparar os valores e retornar o maior valor encontrado. A atividade realizada engloba desde o planejamento, procedimentos requeridos, prática e elaboração do relatório contendo informações sobre como foi construído o trabalho.

2 DESENVOLVIMENTO

O desenvolvimento do projeto iniciou-se com o uso do site Replit para codificação e realização dos testes. Entretanto a realização da atividade pela ferramenta exigida se tornou inviável, pois a plataforma atualmente foge do que precisa para conclusão da tarefa, sendo preciso pagar para ter acesso ao necessário. Em seguida, foi adotado o uso do VS Code, considerando que a ferramente atende todas as necessidades precisas para realização da atividade de forma simples e rápida.

2.1 MÉTODO

Com base na atividade proposta pela disciplina, É realizado os testes de caixa branca, também conhecidos como testes estruturais ou de código, visando avaliar o sistema interno e seus comportamentos, observando a lógica e fluxo de dados. Na etapa de desenvolvimento, empregou-se o código base com algumas melhorias na estrutura e lógica dele, para realização das atividades de testes.

2.2 RESULTADOS

A Figura 1 apresenta o código, aqui se nota a função máximo, onde é recebido as variáveis x e y e realiza verificação se é maior ou não. Na estrutura de repetição com a condição verdadeira é pedido os valores numéricos e saindo dessa estrutura apenas quando tiver os dois valores, um adendo é que nessa mesma estrutura é realizado o tratamento de exceções, que no caso se digitado caracteres retorna a mensagem de erro. Na penúltima e ultima linha é onde se tem os valores armazenados e exibidos com o resultado da operação.

Figura 1 – Imagem do código

maior_ou_menor.py > ...

```
1  def maximo(x, y):
2      if x > y:
3          return x
4      else:
5          return y
6
7  while True:
8      try:
9          print("--------------------------------------------")
10         x = int(input("Insira o primeiro valor: "))
11         y = int(input("Insira o segundo valor: "))
12         print("--------------------------------------------")
13
14         break
15
16     except ValueError:
17
18         print("*********************************************")
19         print("** Erro: Insira valores numéricos válidos! **")
20         print("*********************************************")
21
22 resultado = maximo(x, y)
23 print("O número {} é o maior".format(resultado))
```

Fonte: Elaborado pelo autor

A Figura 2 apresenta o resultado do primeiro teste, o primeiro teste trata-se de receber os dois valores, sendo o primeiro menor que o segundo e assim retornando o resultado maior. O resultado desse teste foi o esperado pelo enunciado do portfólio.

Figura 2 – Imagem do primeiro teste

```
(.venv) lord@GuiPC:~/Downloads/Python/Testes_de_Software$ python3 maior_ou_menor.py
-----------------------------------------
Insira o primeiro valor: 10
Insira o segundo valor: 20
-----------------------------------------
O número 20 é o maior
(.venv) lord@GuiPC:~/Downloads/Python/Testes_de_Software$
```

Fonte: Elaborado pelo autor

A Figura 3 apresenta o resultado do segundo teste, o segundo teste é semelhante ao primeiro, apenas invertendo a ordem de entrada de valores. Sendo assim, o primeiro valor será o maior e o segundo menor. O segundo teste atende os critérios exigidos pelo projeto.

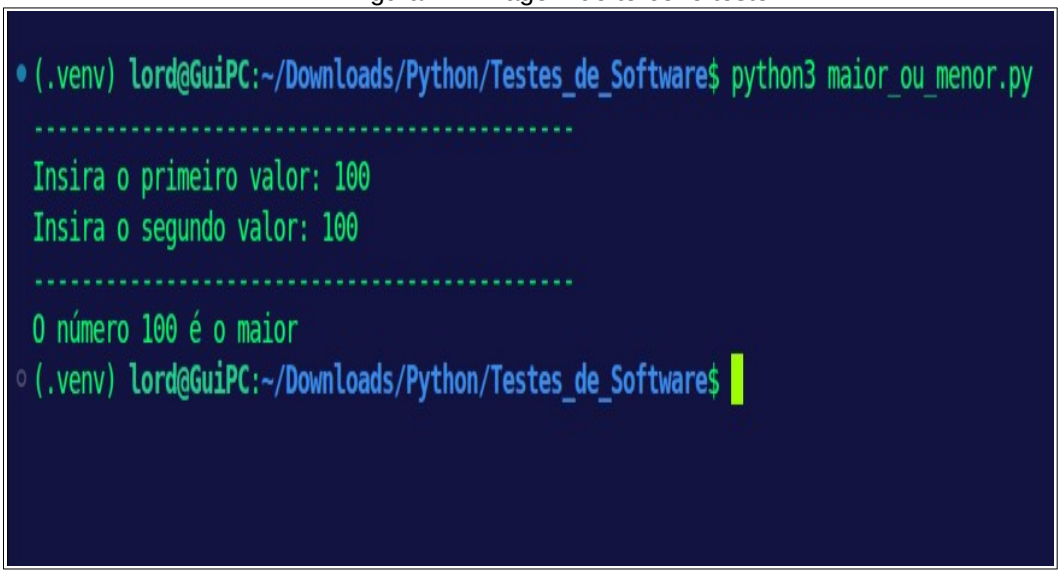
Figura 3 – Imagem do segundo teste

```
(.venv) lord@GuiPC:~/Downloads/Python/Testes_de_Software$ python3 maior_ou_menor.py
-----------------------------------------
Insira o primeiro valor: 20
Insira o segundo valor: 10
-----------------------------------------
O número 20 é o maior
(.venv) lord@GuiPC:~/Downloads/Python/Testes_de_Software$
```

Fonte: Elaborado pelo autor

A Figura 4 apresenta o resultado do terceiro teste, o terceiro teste é a inserção de valores iguais em ambos campos, tendo como retorno um dos valores. Esse teste é o mais simples, porém o projeto como todo tem uma ótima escalabilidade, ou seja é possível implementar funcionalidades mais complexas que visem uma resposta mais interativa e boas práticas de desenvolvimento.

Figura 4 – Imagem do terceiro teste



Fonte: Elaborado pelo autor

A Figura 5 apresenta o tratamento de um erro crítico. Um erro simples que pode quebrar todo o código, é o caso da inserção de caracteres em campos numéricos de valores inteiros, no lugar de resultar em um erro crítico e ser necessário reexecutar o código, o usuário recebe uma mensagem personalizada sobre o erro, depois é requerido o valor novamente. Graças a estrutura de repetição com a condição verdadeira implementada é possível manter o código em execução até obtenção dos dados desejados, juntamente com isso entra o tratamento de exceções que visa ter mensagens personalizadas para o usuário compreender e não fugir do objetivo do código.

Figura 5 – Imagem de tratamento de erro crítico

```
(.venv) lord@GuiPC:~/Downloads/Python/Testes_de_Software$ python3 maior_ou_menor.py
-----------------------------------------
Insira o primeiro valor: Teste de erro
*****************************************
** Erro: Insira valores numéricos válidos! **
*****************************************

-----------------------------------------
Insira o primeiro valor:
```

Fonte: Elaborado pelo autor

3 CONCLUSÃO

O desenvolvimento deste projeto possibilitou a aplicação prática dos conceitos relacionados aos testes de software, com ênfase no teste de caixa branca. A atividade contribuiu para a compreensão da complexidade da área, bem como para o reconhecimento das diversas ferramentas disponíveis para automação de testes em diferentes escalas de aplicação.

Destacam-se ferramentas como PyTest, DocTest e Unittest, que permitem a automatização de testes, variando desde recursos nativos da linguagem Python até bibliotecas externas especializadas. Essas ferramentas diferenciam-se quanto às funcionalidades oferecidas e ao nível de complexidade exigido para projetos de maior porte.

REFERÊNCIAS

PYTHON. **unittest — Unit testing framework — Python 3.8.2 documentation**. Disponível em: <https://docs.python.org/3/library/unittest.html>.

PYTEST. **pytest: helps you write better programs — pytest documentation**. Disponível em: <https://docs.pytest.org/en/stable/>.

BYLEARN. **5 Melhores frameworks para testes com Python**. Disponível em: <https://dojo.bylearn.com.br/python/5-framework-para-testes-com-python/>. Acesso em: 23 fev. 2026.

doctest — Test interactive Python examples — Python 3.8.4rc1 documentation. Disponível em: <https://docs.python.org/3/library/doctest.html>.

MAVEGUI. **GitHub - Mavegui/Faculdade_Testes_de_Software: Projeto da disciplina Testes de Software e Manutenção da faculdade em Engenharia de Software.** Disponível em: <https://github.com/Mavegui/Faculdade_Testes_de_Software>. Acesso em: 25 fev. 2026.